RESEARCH-ARTICLE

Analysis of a cross-domain reference architecture using change scenarios

LILIANA DOBRICA

EILA OVASKA, VTT Technical Research Centre of Finland Ltd, Espoo, Uusimaa, Finland

Published: 13 September 2011

ECSA '11: Companion Volume
September 13 - 16, 2011
Essen, Germany

Analysis of a Cross-Domain Reference Architecture using Change Scenarios

Liliana Dobrica
University Politehnica of Bucharest
Bucharest, Romania
liliana@aii.pub.ro

Eila Ovaska
Technical Research Centre of Finland
Oulu, Finland
Eila.Ovaska@vtt.fi

ABSTRACT
The content of this paper addresses the issue of how to perform analysis of a cross domain reference architecture. The cross domain reference architecture is designed based on the domains requirements and features modeling. The definition of a cross domain reference architecture is based on well known concepts from software architecture description, service orientation and product line. We apply a method based on change scenarios to analyze variability at the architectural level. In order to handle complexity in analysis we propose categories of change scenarios to be derived from each problem domain and we provide informal guidelines for each step of the analysis method.

Categories and Subject Descriptors
D.3.3 [**Programming Languages**]: Language Constructs and Features – *abstract data types, polymorphism, control structures.*

General Terms
Design, Measurement, Algorithms, Documentation, Verification.

Keywords
Cross domain reference architecture, service, variability, quality, analysis methods, scenarios.

1. INTRODUCTION
Nowadays the application domains of embedded systems include many systems as subsystems that need to satisfy complex requirements. Due to the escalating complexity level and the higher competition in the world market, a coherent and integrated development strategy is required. A key research challenge remains how to create of a generic architecture and a suite of abstract components with which new developments in different application domains of embedded systems can be engineered with minimal effort. In the last decades many companies have successfully improved productivity, enhanced quality and reduced time to market by adopting Software Product Lines (SPL) [Pohl]. The main goal of SPL is the development of product line software architecture, which is similar to the goal of complex embedded systems. In our research, we propose the development of cross-domain reference architecture (RA) based on a common architectural style supporting the composition of systems out of independently developed components that meet the requirements of various application domains. Given a core architectural style, different components are created for different application domains, while retaining the capability of component reuse across these domains [25].

This paper elaborates on the architectural analysis to get measures of compliance with regard to changing requirements specification [6]. In the case of cross domain embedded systems it is very important to identify which are the relevant properties of each domain and how analysis techniques and methods could be applied to a cross domain reference architecture (RA). There are two categories of properties, the general properties, like performance, satisfaction of real-time requirements, reliability, etc. and specific properties related to a development process [4]. Among the specific properties that deserve special attention are kinds of variation which can be covered by the architecture and properties that are preserved for all variants of an architecture in specific domains, stability of services interfaces with respect to evolution in products.

The open problem of an architectural analysis method is how to take better advantage of architectural concepts to analyze for quality attributes in a systematic way [2][6][7]. In a product line approach, the RA must be generic and adaptable to the multiple composed domains. One objective of the evaluation is to minimize possible changes in functionality required by various domain specific services. It is also very important to identify potential risks and to verify that the quality requirements of the embedded systems domains have been addressed in the RA design [9]. Analysis could be associated with the design in an iterative improvement of the RA when the system of systems is initiated from requirements specification, or for the re-engineering of an existent complex embedded system due to the evolution process.

In this context, in this paper we propose a method that performs analysis of a cross-domain reference architecture for embedded systems applications using change scenarios. In the next section we define the cross domain approach for architecture development, then we discuss about an appropriate quality

ECSA'11 Workshop SAVA'11, September 13, 2011, Essen, Germany.

analysis method. Finally we explain our view on a concrete example and we perform architectural analysis. We discuss related work in section 6, and we conclude the paper in section 7.

2. ARCHITECTURE DEVELOPMENT APPROACH

In our recent work [25] we have addressed the problem of defining of a cross domain approach that extends to three levels the architecture development of a software system in a product line initiative (Figure 1).

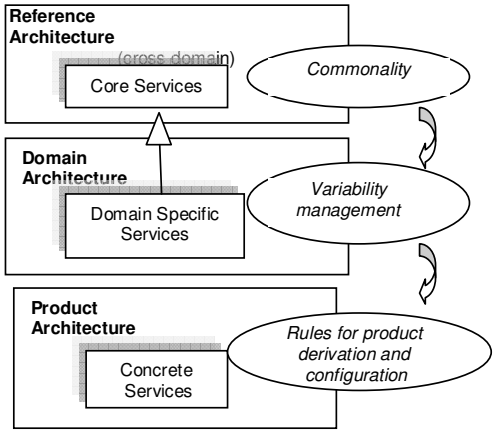

Figure 1. Architecture development in a product line approach.

Thus we consider the software system as a collection of cooperating services that deliver required functionality. These services may be executed in a networked environment and may be recomposed dynamically. The RA level includes core services and focuses on commonality analysis. Also the RA includes rules or constraints on how core services should be combined to realize a functional goal.

Domain architecture level includes domain specific services and requires variability management concerns. Variability management is a key issue in the success of product line development. Approaches based on model driven architecture (MDA) [15] or ontology based support [16] have been identified in our researches. Deriving individual products from shared core software assets is still rather time consuming and expensive for a large number of organizations. A method for building product populations with software components has been applied with success in industries [17]. However various studies and experience reports of the current practice in industry [18,19,20] have identified several problems associated with products derivation. Knowledge externalization, variability management, scoping and evolution are important areas of concerns. Under development are automated approaches [21] or specific tools [22] that still remain immature for industrial processes.

Domain architecture describes ready- made building blocks that assist application/products developers in using specific domains services. When the reference architecture has been defined, the existing components and services are considered as building blocks in the architecture of the set of products. The domain services provides variable assets repository. Variability appears in functional and non-functional requirements (including quality attributes). A structured domain architecture repository may be provided at this level. A schema for this repository has to be defined in a form of relationships between services. In this way we are mapping domain specific services to core abstract services. Specialization relation is a solution to be used for variability management. Modelling run-time quality attributes variability requires tool support. We can mention an existent approach [23] that has been developed and refined. However monitoring mechanisms, measuring techniques and decision models for making tradeoffs should still be defined and validated.

Finally, the last level is dedicated to the set of product architectures. On this level rules for product derivation and configuration are included. Product architecture of a product line consists of concrete services derived and configured based on rules. The goal of product derivation is to reach a configuration of the product family in which necessary variabilites have been bound. The decision model for bounding specific services of a domain to a product may be in a tabular form or a more comprehensive tool based on the feature types and composition rules. By selecting a consistent set of features required for the individual product, the corresponding domain specific services that realize those features are selected from the domain architecture repository to constitute the product.

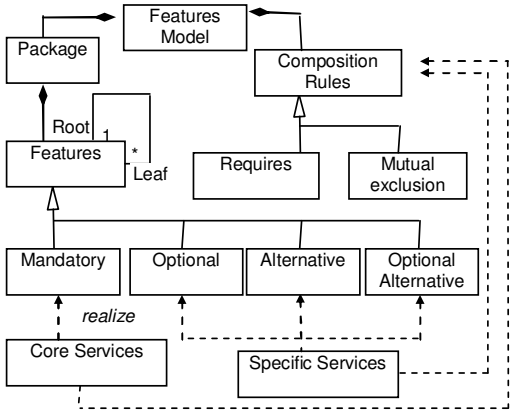

Figure 2. Features – UML metamodel.

A feature model is a prerequisite of our approach [11]. This model is essential for both variability management and product derivation, because it describes the requirements in terms of commonality and variability, as well as defining dependencies. Features may be mandatory, optional, alternative or optional alternative. We have built a UML meta-model for features modelling (Figure 2). The features model specifies dependencies called composition rules. The *requires rule* expresses the presence implication of two features and the *mutually exclusive*

rule captures the mutual exclusion constraint on feature combinations.

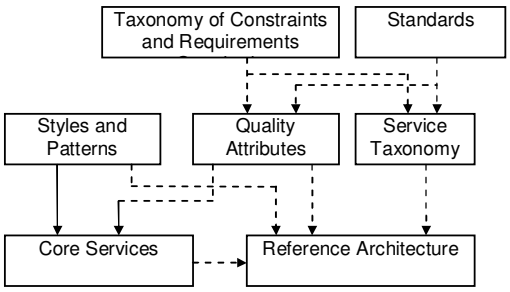

Figure 3. Reference architecture realization.

RA defines quality attributes, architectural styles and patterns and abstract architectural models (Figure 3). *Quality attributes* clarify their meaning and importance for core service components. Services have to meet many quality attributes, such as modifiability or integrability. Modifiability of a service can be divided into the ability to support new features, simplify the functionality of an existing service, adapt to new operating environments, or restructure system services. Integrability measures the ability of the parts of a system to work together. It depends on the external complexity of the components, their interaction mechanisms and protocols, and the degree to which responsibilities have been cleanly partitioned. Interdependencies and tradeoffs also exist between quality attributes.

Styles and patterns are the starting point for architecture development. Architectural styles and patterns are utilized to achieve qualities. A style defines a class of architectures and is an abstraction for a set of architectures that meet it. An architectural pattern is a documented description of a style or a set of styles that expresses a fundamental structural organization schema applied to high-level system subdivision, distribution, interaction, and adaptation [5]. Design patterns, on the other hand, are on a lower level. They refine single components and their relationships in a particular context [8]. In this way the RA creates the framework from which the architecture of new products is developed. It provides generic architectural services and imposes an architectural style for constraining specific domain services in such a way that the final product is understandable, maintainable, extensible, and can be built cost-effectively. Potential reusability is highest on RA level. Core services and the architectural style are reused in every domain.

RA is build based on a *service taxonomy*. We adopted the idea from WISA [10] of an existing knowledge on software engineering that is integrated and adapted to service engineering. The standards related to each domain, applicable architectural styles and patterns and existing concepts of services and components are the driving forces of development. A service taxonomy defines the main categories called domains. Typical features that have been abstracted from requirements characterize services. A service taxonomy guides the developers on a certain domain and getting assistance in identifying the required supporting services and features of services.

3. ANALYSIS METHOD

Scenario-based assessment is particularly appropriate for qualities related to software development, which are specific to product line approach. Software qualities such as maintainability, reusability, modifiability, adaptability and portability can be expressed very naturally through change scenarios. In [1] the use of scenarios for evaluating architectures is recommended as one of the best industrial practices [13]. Our method is based on the SAAM [12], but improved through the introduction of guidelines for analysis and categories of change scenarios. The goal is to establish how adaptable the PLA is to the expected changes related to this taxonomy. The analysis considers PL specific techniques such as commonality analysis, which systematically models the required similarities and differences among PL members. It is also considered that PLA contains the common components of the architectures of the product members and takes variabilities as possible changes to this.

The analysis method consists of five important steps, which are: *deriving of change categories from the problem domain, scenarios identification, RA description, evaluation of the effect of the scenarios on the architecture elements, scenario interaction.* A description of each one:

1. Deriving of change categories from the problem domain. Figure 4 presents five categories of the change scenarios derived from the problem domains. These categories can be refined into more specific categories to facilitate the next step of identifying scenarios. For example a "Domain specific software" sub-category refines "Software technology" category. Also there are systems where "Environment" change category need to be considered.

Guideline: It may be possible that a change scenario related to one of these categories requires other changes in the other categories. It is recommended to consider this possibility in the scenario development process. Usually it occurs when the problem domain is organized so that it is easy to identify the main sources for the addition of subsequent features in the domain.

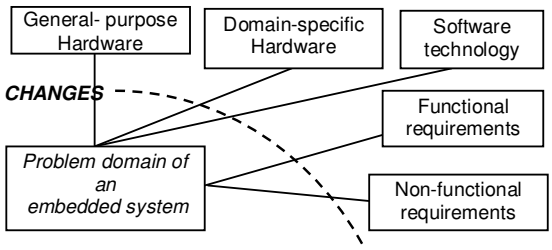

Figure 4. Categories of scenarios.

2. Scenarios identification. In this activity we distinguish possible changes that may happen in the life of the system based on the derived categories. Scenarios should illustrate the kinds of anticipated changes that will be made to the system. These changes may result from a natural maturation of hardware or software technology or from the introduction of new functional or non-functional requirements.

Guideline 1: A common problem of the scenario development is when to stop generating scenarios. For example, using a set of *standard quality attribute-specific questions* we ensure proper coverage of an attribute by the scenarios. The boundary conditions should be covered. A standard set of quality-specific questions allows the possibility of extracting the information needed to analyze that quality in a predictable, repeatable fashion. We consider that the architecture is a good one and it is not necessary to generate scenarios to verify the functional requirements. Otherwise these should also be considered when verifying functionality. For analyzing the modifiability we must look for possible changes in the problem domain defined requirements.

Guideline 2. We introduce the definition of the type of a scenario. A type could be simple or combined, where it comprises a combination of simple types. A simple type represents a) a possible situation in which an existent service evolves based on changes in functionality or apart from the change of functionality, there are changes in its interfaces. b) a possible situation in which a new variant of service is introduced, thus the feature model of a product must be extended with a new variant; c) a possible situation in which a product feature tree is modified because of changing a feature from an alternative to mandatory; d) a possible situation that requires changes of a product environment. The a simple type can affect a single variant of a product and a combination if multiple simple types of scenarios can affect multiple products.

3. Architecture Description could be performed in parallel with the previous one. Architecture description may use multiple views. For a common level of understanding a small and simple lexicon could be used in describing structures. It is important to ensure consistency within each view and among multiple views by using consistency rules.

4. Evaluate the effect of the scenarios on the architecture elements. The effect is estimated by investigating which services are affected by that scenario.

Guideline: The cost of the modifications associated with each change scenario is predicted by listing the services that are affected and counting the number of changes. The objective is to get a measurement of the quality of the core and domain services with respect to the anticipated variability in hardware or software technologies, functional or non-functional requirements.

5. Scenario interaction. The result of the effects evaluation represents the input for this step. The activity is to determine which scenarios interact, meaning that they affect the same service. High interactions of unrelated scenarios indicate a poor separation of concerns.

Guideline: A special attention should be paid to possible situations of dynamic modifications of deployed products, with minimal interruption of services, where the goal is to minimize the number of changes required to migrate each product. If any of the scenarios affect a core service this is a situation where all products require an update. However, in practice, often only a limited set of products have to be updated. This evaluation can be automated if three important information exist, which are the feature configuration used to derive a product, the information about the variability at the time of product derivation and if necessary the product relevant description of the environment to which a service-based product is deployed.

4. THE CASE STUDY

An analysis method is very difficult to discuss on an abstract level. Instead, one needs a concrete example. In this section we present a case study of cross domain reference architecture. Our example is abstracted from our experiences with the architecture design of a scientific on-board silicon X-ray array (SIXA) spectrometer control software. SIXA is a multi-element X-ray photon counting spectrometer. It consists of 19 discrete hexagonally-arranged circular elements and specific domain hardware architecture. The measurement activity consists of observations of time-resolved X-ray spectra for a variety of astronomical objects.

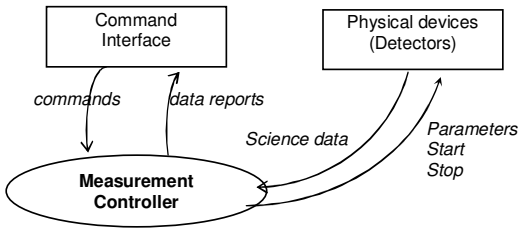

Figure 5. Context view of the required system.

Figure 5 introduces the context view of a measurement controller, which describe relations between the system under analysis and its interactions with the environment. External elements that interface with our measurement controller are a command interface and physical devices (detectors) representing sensors and actuators.

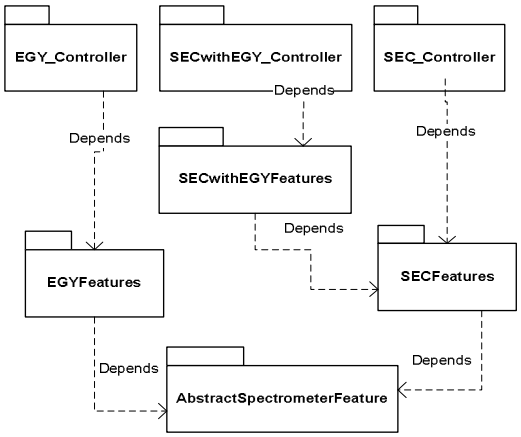

Figure 6. Mapping features into packages.

The embedded system is programmed and operates using a set of commands sent from a command interface. The role of the spectrometer controller is to control the following measurement modes: (a) Energy Spectrum (EGY), which consists of three energy-spectrum observing modes: Energy-Spectrum Mode (ESM), Window Counting Mode (WCM) and Time-Interval Mode (TIM). (b) SEC, which consists of single event characterization observing modes: SEC1, SEC2 and SEC3. Each

mode could be controlled individually. A coordinated control of the analog electronics is required when both measurement modes are on. The analysis of requirements for domain engineering has a result in a features model. The features model has been structured in packages (Figure 6). The *with reuse* aspect of reusability is described in the architecture by the abstract features.

The abstract features encapsulated in three main abstract domains MeasurementController, DataManagement and DataAcquisition, are completely reused in all the derived products. The AbstractSpectrometerFeatures package has the highest degree of reusability but also the highest degree of dependability. The abstract features depend on the commonality between EGY and SEC features. A change in the problem domain of one of the three products is mostly reflected in the degree of reusability of the abstract domain features.

The sets of products that could be derived from the domain specific services during application engineering are: (1) P1 – EGYController includes specific services of a standalone control of EGY mode; (2) P2 – SECController includes specific services of a standalone control of SEC mode; (3) P3 – SECwithEGY Controller includes specific services of coordinated control.

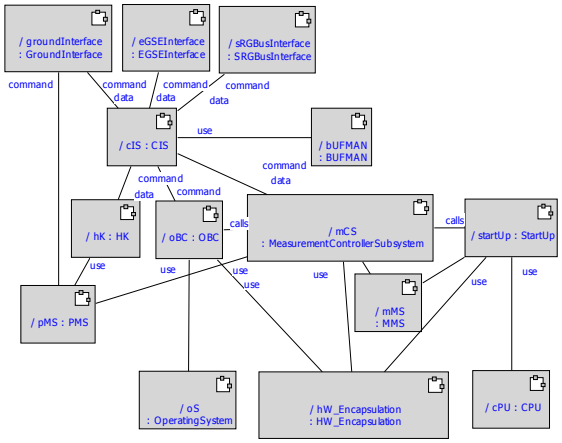

Figure 7. Conceptual view.

The architecture model is documented around multiple views describing conceptual and concrete levels, for each view a static and dynamic perspective being offered. Architecture documentation addresses specific concerns for measurement control, data acquisition control and data management. The views are illustrated with diagrams expressed in UML-RT, a real-time extension of UML. The conceptual level considers a functional decomposition of the architecture into domains. The concrete level considers a more detailed functional description, where the main architectural elements are packages, capsules, ports, protocols. In order to predict any quality attribute based on the architecture model it is important to define a clear and precise terminology of the elements used for description:

Conceptual View is the result of a functional-based decomposition, and it includes relations between different domains. The architectural components are large functional (domain) entities, and the connectors are *"uses", "command" or "passes-data-to"* relations. This structure is useful for understanding the interactions between entities in the problem space, for understanding the cross-domain perspective, and hence thereafter, the possibilities for creating a system of systems. Conceptual view is illustrated in figure 7. It includes: (1) Measurement Controller Subsystem (MCS) which has the main role in controlling acquisition and dumping science data. (2) Housekeeping (HK) forms the reports and sends them to command interface when requested the command interface subsystem. It uses services provided by PMS. (3) Command Interface Subsystem (CIS) hides the hardware buses' interfaces from the rest of the software. (4) On-board clock (OBC) maintains an on-board clock used for time-stamping spectra in data files. It includes services for timing the start/stop of spectra and targets and other timing related services. (5). Memory Management Subsystem (MMS) provides services for handling the storages in RAM and EEPROM areas. (6) Parameter Management Subsystem (PMS) provides services for initiating, changing and reading the on-board parameters in EEPROM. (7) StartUp implements the power up and watchdog timer start-up. (8) Communication buffer management (BUFMAN) provides services for allocating/deallocating transmit buffers. (9) CPU specific services provides highly optimized high speed assembly language services (high speed word copy, interrupt enable/disable). (10) Hardware encapsulation modules control specific hardware (analog electronics, watchdog timer).

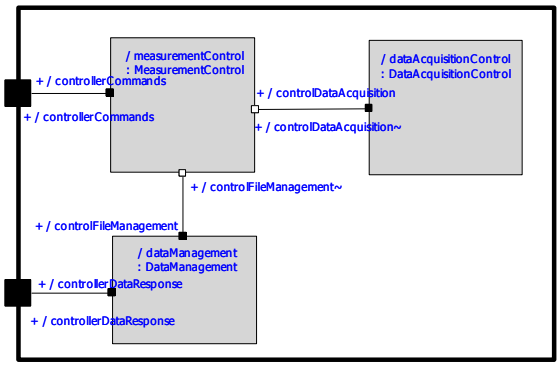

Figure 8. Detailed Functional Decomposition.

In a detailed functional decomposition view the main elements are packages, components, ports and protocols. The static relations between components are association, specialization, generalization, etc. Considering the dynamic relations, statechart diagrams and sequence diagrams are also part of this view. In this view abstract components are included (Figure 8) based on a recursive control architecture style [14]. The MeasurementControl component, which is responsible for starting and stopping the operating mode for data acquisition according to the commands received from the command interface, and according to the events generated in other parts of the software. The DataAcquisitionControl component collects events (science data) to the spectra data file during the observation of a target. The component includes as well as hides data acquisition details. DataManagement provides interfaces for storing science data - opening/ closing/writing the data files, hiding data storing details. The component also provides interfaces for controlling the transmission of stored data to the command interface.

5. ARCHITECTURE ANALYSIS

5.1 Change Scenarios Effects

We define twelve change scenarios for different categories of changes as: scenario no. 12 is for general-purpose hardware changes; scenarios no. 2,3,4,7 and 8 are for domain-specific hardware changes; scenario no. 5 is for technology changes; scenario no. 11 is for non-functional requirements changes; and scenario no. 1 for other changes.

The effect of the scenarios and the required views are analyzed in the following.

1: Receive the target coordinates from a STAR TRACKER (ST).

Effect on the architecture: The target coordinates received from a ST will overwrite the coordinates given previously with the COORD-ground command, which contains the pre-plannedcoordinates. This requires a new protocol to the MeasurementControl domain to interface with the ST. The protocol is included in the concrete level, because it is a common feature of the two concrete components; StandAloneControl and CoordinatedControl.

Result: Modify one component in the detailed functional decomposition view.

2: Change SIXA parameters (i.e. change the number of detectors).

Effect on the architecture: This is a hardware modification scenario, which is reflected in the CPAR command. The measurement control subsystem uses the service of PMS. This scenario requires modification in PMS, in this way the measurement control subsystem is decoupled from changes in the hardware-specific domain.

Result: Modify one component in the conceptual view.

3*:* Change the number of EEPROM banks.

Effect on the architecture: This is a hardware-specific change scenario. The program code is stored in the EEPROM memory, which is organized into 4 banks. The read memory command contains the representation information of the physical EEPROM addresses. The measurement control subsystem is decoupled from the memory organization by StartUp, which executes the command.

Result: Modify one component in the conceptual view.

4: Add a hard disk for SEC product.

Effect on the architecture: The SEC_controller and SECwithEGY_controller contain a HD for data storage. This scenario requires a lot at the architectural level, most of them related to the Data Management component.

Result: Multiple changes in detailed functional decomposition, localized in the SECFeatures.

5: Change the generator polynomial (from CCITT polynomial) for 16 bit CRC sum of errors handlers.

Effect on the architecture: MMS consists of service functions for managing the storage RAM and EEPROM. It also includes a state for refreshing RAM and the memory error exceptions handlers (double and single bit).

Result: Modification in one component in the conceptual view.

6: Add a new command – SetDiskFull – in service commands group of ground interface.

Effect on the architecture: The command is specific to the software configuration with the hard disk. The measurement control component interprets this new command and starts the required action. This scenario requires the addition of a new signal to the controller commands protocol and data management protocol. A new transition state transition in the Data FileManagement component.

Result: Changes in detailed decomposition view.

7: Specific hardware configuration (hard disk capacity or RAM capacity) changes.

Effect on the architecture: If the capacity of the hard disk or the amount of fault-free RAM in use for SEC spectrum storage is changed, then the calculation of the SEC event limit used in SEC1 and SEC2 modes is changed.

Result: This scenario requires no change at the architecture level.

8: The type of hard disk is changed.

Effect on the architecture: This scenario is applicable for the software products with SECFeatures included. Reading data from the disk during dumping is time-critical because the average SRG-bus speed must be maintained. The disk driver must be optimized so that it makes maximum use of the internal cache.

Result: Modify one component in conceptual view.

9: How is the control of analog electronics changed on behalf of several spectrometers?

Effect on the architecture: In case of several spectrometers, only one of them takes control over the analog electronics. This feature is included in the CoordinatedControl component.

Result: Modification of one component in detailed functional decomposition.

10. How is the architecture affected when the operation mode is changed?

The operation modes is one of the variability among domains. This is encapsulated into DataAcquisition and DataFileManagement.

Effect on the architecture: The measurement control component is decoupled from the operation mode of different products, which is encapsulated into the DataAcquisition component.

Result: No change to the RA – abstract concrete or features of measurement control.

11. How is the average SRG-bus speed of 744kbit/sec on reading data from disk, which is time critical, maintained?

Alternative solutions: (1) Change the HD: Use Fast disk: Optimal disk interleaving factor and storing the data file in

sequential sectors on the disk. (2) Send filler blocks to the bus while waiting for the disk – a sufficient number of filler blocks could be reserved in the vector word sent in advance to BIUS. (3) Use a busy bit of SRG-bus. (4) Optimize disk driver – If the disk drive has been changed, the software has to be tuned separately for the new disk.

Result: Not applicable to the available views.

12: Change the CPU.

Effect on the architecture: CPU specific services provide highly optimized high speed assembly language services (high speed word copy, interrupt enable/disable, etc.) The services are not applicable at the level of description.

Result: Not applicable to the available view.

5.2 Detailed analysis of scenarios categories related to hard disk

The hard disk is a specific feature of the SEC_Controller spectrometer domain. Two scenarios could be considered in the category of hardware changes. The addition of the hard disk and the change of the type of hard disk.

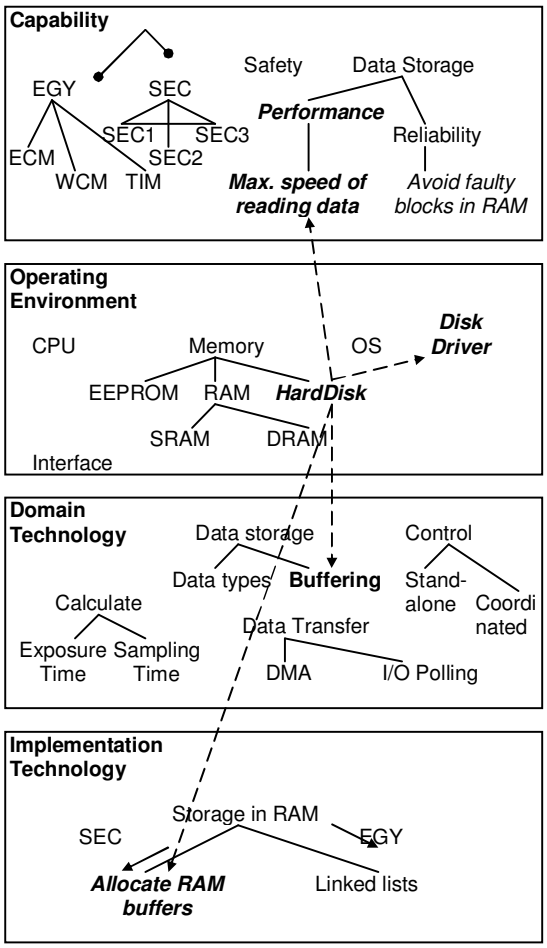

Figure 9. Dependencies between features.

The addition of the hard disk scenario analysis. In the embedded systems there is a very tight relation between hardware and software, and changes in the hardware context influence most of the architectural decisions. The hard disk provides one of the variables in the spectrometer controller specific domain. The addition of the hard disk requires four other new features to be considered in the tree (Figure 9). These are: (1) An Disk Driver in the same category, (2) A feature for maximization of speed in reading data in the capability category, (3) Buffering for data storage in the domain technology category, (4) Storage of data in RAM at successive locations in the implementation technology category. From the conceptual viewpoint all these new features must be mapped in the SEC_Controller package to different diagrams of components. Thus the addition of the hard disk requires: (1) Adding a DiskManagement component in data management, (2) Adding a new protocol, DiskControl, for the connection between SEC_File_Management and Disk_Management components, (3) Addition of a new signal in the ControllerCommands: SetDiskFull – a new command in service commands (abnormal situation) group, and (4) A new transition in the MeasurementControl state diagram.

The specific domain services of Data Management for the products with SEC Features or EGY features are considered. For EGY products the data management service encapsulates only the FileManagement component, which uses a saved_EGYData protocol. The SECData Management component encapsulates two components, SEC_FileManagement and Disk Management. These components use saved_SECData and ControlDisk protocols.

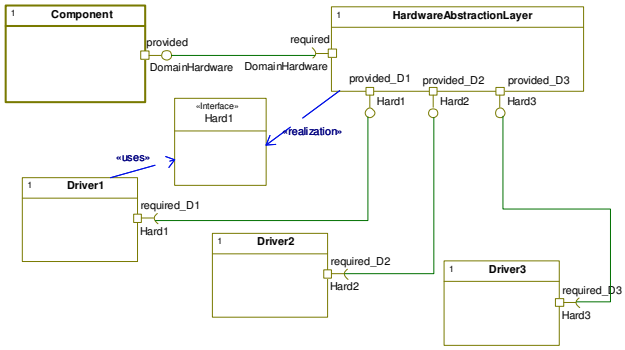

Figure 10. Decoupling of Component software architecture from hardware.

The change of the type of hard-disk. In the interface with hardware the description of the architecture includes the mechanism of mapping virtual information which is used in software to physical hardware. It is the role of a hardware abstraction layer to perform this operation (Figure 10). The mechanism role is the decoupling of software from hardware. The component whose interface handles the variability aspects of hardware elements is decoupled from this variability by introducing a hardware abstraction layer. A disk driver component, as part of the hardware abstraction layer, has been introduced at once with the hard disk. The change of the hard disk type is localizing in this component.

6. RELATED WORK

This section discusses about the relevance of our work in comparison with other related research. In the last decades multiple software architecture analysis methods were proposed. The first survey addressing architectural analysis came with the comparison framework of the methods [6]. Our work is based on this framework to elaborate on the guidelines for help to achieve the objectives of each step. However the present work includes guidelines related to variability, which could be a starting point towards automation of the analysis. A change impact analysis in the context of assessing evolution has been discussed in [24]. Here scenarios are used to identify affected products and a change impact analysis model is defined. This approach does not consider categories of scenarios, but it distinguishes between simple and combined types of scenarios. Neither has it identified dependencies of scenarios.

In paper [26] the authors describe an approach that integrates feature modeling techniques, meta-modeling, multiple-view modeling and service modeling to model the views pertinent to service oriented architectures variability. The approach in this paper is similar to our approach in that it applies the same concepts and it addresses only design time variability and not runtime variability issues.

7. CONCLUSION

In this paper, we elaborated on the problem of analysis of a cross-domain RA using change scenarios. The discussion is illustrated with an example from embedded systems application domain. A good architecture design must provide a good localization of changes. Most of the changes required by scenarios were applied to one component, which indicates a good decoupling of concerns. The most important change was the addition of the hard disk, a variability among domains. This scenario requires changes to the domain specific components. By structuring the RA in abstract services, which encapsulate abstract features of the domains and concrete components, which in turn represents specialization of the variable features, the effects of the change scenarios are minimized and localized.

For the moment, only change scenarios could be used in RA analysis in early stage of development. One problem with scenario-based analysis is that the result and the expressiveness of the analysis are dependent on the selection of the scenarios and their relevance for identifying critical assumptions and weaknesses in the architecture. There is no fixed minimum number of scenarios whose evaluation guarantees that the analysis is meaningful. According to this, we tried to use five categories of possible changes in general hardware, specific hardware, functionality, non-functional requirements and software technology. A helpful strategy in scenario elicitation is the analysis of commonality and variability. This is not a part of the analysis method, but it is considered a pre-condition of it. One aim of the analysis should be to show how flexible a RA is in order to handle the anticipated changes provided by the variability of domains. Another aim is to analyze which is the potential of the RA to be adapted to changes in common features.

The results of the analysis depend not only on the views of the architecture, but also on the level of detail of the services descriptions. By using only the conceptual view the effects of the change scenarios are reduced. On the detailed functional decomposition view, which has been developed with the help of a CASE tool, the effect is more relevant. The interaction of unrelated scenarios is lower and it reveals a good separation of concerns when the domains decomposition is detailed.

In this work we introduced our approach for architecture development and we detailed the description of the analysis method. We are aware that the current version of our method does not address several issues and we hope that a discussion of the proposed method can contribute to its refinement.

In our future research, we plan to refine the method. For example, we want to apply the method to a number of other case studies from other problem domains and to refine the generic categories of scenarios into more specific categories to facilitate the task of identifying scenarios. Under refinement are also guidelines for identifying changes in a category of scenarios from the identification of changes in another category and to identify the scenarios most relevant to the identification of points of variability.

8. ACKNOWLEDGMENTS

This work was supported by CNCSIS –UEFISCSU, project number PNII – IDEI 1238/2008.

9. REFERENCES

[1] M. Barbacci, M. Klein and C. Weinstock, 1997 *Principles for Evaluating the Quality Attributes of a Software Architecture.* Technical Report, CMU/SEI-96-TR-036..

[2] Bass L., P. Clements, R. Kazman. 2003, *Software Architecture in Practice*. 2nd ed., Addison-Wesley.

[3] Bengtsson PO., J. Bosch, 2004, Architecture Level Modifiability Analysis, Procs of the ICSR5.

[4] Bosch J, 2000, Design and Use of Software Architectures - Adopting and Evolving a Product-Line Approach, Addison Wesley.

[5] Buschmann F., R. Meunier, and H. Rohnert, 1996, Pattern-Oriented Software Architecture:A System of Patterns. John Wiley and Sons.

[6] Dobrica L., E. Niemelä, 2002, A Survey on Software Architecture Analysis Methods, IEEE Trans on Soft. Eng, vol 28(7).

[7] Dobrica L., E. Niemelä, 2008, Quality and Value Analysis Method for Product Line Architectures, ICSOFT 2008.

[8] Gamma E., R. Helm, R. Johnson, and J. Vlissides, 1994, Design Patterns: Elements of Reusable Object-Oriented Software, Addison Wesley.

[9] Graaf B, vanDijk H, van Deursen A., 2005, Evaluating an embedded software reference architecture – industrial experience report, CSMR 2005, 354-363.

[10] Niemelä E, Kalaoja J, Lago P, 2005, Towards an architectural knowledge base for wireless service engineering. IEEE Trans. on Software Engineering, vol. 31 (5), 361 – 379.

[11] Kang, K. C., Kim, S., Lee, J., Kim, K., 1998, FORM: A Feature-Oriented Reuse Method with Domain-Specific Reference Architectures. Annals of Software Engineering, 5, Pp. 143-168.

[12] Kazman R., G. Abowd, L. Bass, P. Clements, 1996 *Scenario-based Analysis of Software Architecture*. IEEE Software, pp. 47-55.

[13] Kazman R., S. J. Carriere and S. G. Woods, 2000, Toward a Discipline of Scenario-Based Architectural Engineering, *Annals of Software Engineering*, Vol. 9.

[14] Selic B., Recursive control, in: R. Martin, et al. (Eds.), Patterns Languages of Program Design—3, Addison-Wesley, 1998, pp. 147–162.

[15] Santos A. et.al., An MDA Approach for Variability Management in Product-Line Engineering, Nord. J. Computing, vol 13, 196-213.

[16] Mohan K. and Balasubramaniam R., Ontology-based Support for Variability Management in Product and Service Families, HICSS, 2003.

[17] Ommering, R. van, 2002. Building Product Populations with Software Components, Proceedings of the International Conference on Software Engineering 2002, 255–265.

[18] Deelstra, S., Sinnema, M., Bosch, J., Product derivation in software product families: a case study, JSS 74, 173-194, 2005.

[19] Graaf B, vanDijk H, van Deursen A., Evaluating an embedded software reference architecture – industrial experience report, CSMR 2005, 354-363.

[20] Ho-Sung H. et al., An Embedded Software Architecture for Advancing Modifiability in Digital Convergence Environment, SERP 2007.

[21] Gomaa H., Shin M., Automated Software Product Line Engineering and Product Derivation, HICSS 2007.

[22] Haugen O et al, A MDA-based framework for model-driven product derivation, IASTED Software Engineering. and Applications, 2004.

[23] Niemelä E., Evesti A., Savolainen, P, Modeling Quality Attribute Variability, ENASE 2008.

[24] Michalik B., D. Weyns, Towards a Solution for Change Impact Analysis of Software Product Line Products, 2011 Ninth Working IEEE/IFIP Conference on Software Architecture, pp. 290-294.

[25] Dobrica L., Ovaska E., Service Based Development of a Cross Domain Reference Architecture, ENASE 2008/2009, CCIS 69, pp. 305-318, 2010, Springer.

[26] Abu-Matar M., H. Gomaa, Feature based Varibility for Service Oriented Architectures, 2011 Ninth Working IEEE/IFIP Conference on Software Architecture, pp. 302-310.